Cloud Data Fusion

A Practical Handbook for Visual ETL/ELT on Google Cloud

What Cloud Data Fusion is, how its point-and-click pipeline builder actually works under the hood, every technical term explained in plain English, and a full walk-through demo you can run yourself end to end — from a raw CSV in Cloud Storage to a clean table in BigQuery.

Demo included	Section 7 — build and run a full Wrangler + Pipeline Studio job, GCS to BigQuery
Primary services	Cloud Data Fusion · Cloud Storage · BigQuery · Managed Service for Apache Spark (formerly Datap
Audience	Data engineers and analysts new to Cloud Data Fusion; trainers running a live demo
Demo footprint	Single Developer-edition instance, one small CSV, one pipeline — delete the instance when you're d

Contents

1. What Is Cloud Data Fusion, and What Problem Does It Solve?

Building a data pipeline the traditional way means writing and maintaining code: a Spark job to read a source, a series of transformation steps, a writer for the destination, plus retry logic, schema handling, and logging around all of it. That's fine when you have engineers to spare, but most organizations have far more integration work — connecting this database to that warehouse, cleaning up that vendor's export format — than they have engineering hours.

Cloud Data Fusion is Google Cloud's fully managed, **visual** data integration service. Instead of writing code, you build a pipeline by dragging boxes onto a canvas — a source box, a few transformation boxes, a destination box — connecting them with arrows, and configuring each one through a form. Under the hood, Cloud Data Fusion compiles that visual pipeline into an actual Spark job and runs it on managed infrastructure that spins up only for the duration of the run.

It is built on top of **CDAP** (the Cask Data Application Platform), an open-source framework for building and running data applications. Google took CDAP, made it cloud-native, and wrapped it in a managed service — meaning you get CDAP's pipeline model and plugin ecosystem without having to install, patch, or scale any of it yourself.

1.1 Why not just write a Spark or Dataflow job?

- **Speed of delivery.** A pipeline that would take a day to code, test, and deploy can often be built in Cloud Data Fusion's Studio in under an hour, because most of the common logic — reading a format, filtering rows, joining datasets, writing to a warehouse — already exists as a pre-built, configurable plugin.
- **Accessibility.** Analysts and less code-heavy data practitioners can build and maintain real production pipelines, not just engineers — the visual canvas is the documentation.
- **Built-in operational features.** Scheduling, retries, logging, monitoring dashboards, and data lineage tracking come with the platform, rather than being something you build yourself around a hand-written job.
- **A large plugin ecosystem.** Hundreds of connectors exist for common systems — relational databases, SaaS APIs, SAP, mainframes, cloud storage — many of which would be significant engineering effort to hand-write and maintain.

The trade-off: highly custom logic that doesn't fit the plugin model (a very particular transformation, an unusual source protocol) can be more awkward to express visually than in code — though Cloud Data Fusion has an escape hatch for this too, in the form of custom Python/JavaScript transform plugins you can drop into a pipeline (see Section 5).

Where this fits among Google Cloud's data tools

Cloud Data Fusion focuses on **visual, plugin-based batch and streaming ETL/ELT**. It's a different tool from Datastream (which is specifically for log-based Change Data Capture — see the companion handbook on CDC) and from Dataflow (a code-first, Apache Beam-based processing service). Cloud Data Fusion pipelines commonly use Datastream as one of many possible sources, and under the hood execute on the same Spark infrastructure that also powers Dataproc workloads.

2. Core Terminology, Explained Simply

Read this section once, slowly — every later section in this handbook, and the demo in Section 7, assumes you know these words.

CDAP

The open-source data application platform (Cask Data Application Platform) that Cloud Data Fusion is built on. CDAP defines the pipeline model, the plugin interface, and the UI concepts (Studio, Wrangler, Hub) that Cloud Data Fusion exposes as a managed service.

Instance

A dedicated, isolated deployment of the Cloud Data Fusion control plane for your project — effectively, your own private copy of the CDAP UI and API server. You create an instance, then build and run pipelines inside it. Instances come in Developer, Basic, and Enterprise editions (Section 4).

Namespace

A logical grouping within an instance that isolates pipelines, datasets, and configuration from each other — similar in spirit to a project or folder. Most simple setups use the single provided default namespace.

Pipeline

A directed graph of connected stages — sources, transforms, sinks, and so on — that defines an end-to-end data flow. A pipeline can be **batch** (runs to completion on a schedule or trigger) or **streaming** (runs continuously against a live data feed such as Pub/Sub or Kafka).

Pipeline Studio

The visual canvas where you build pipelines: drag a plugin from the left-hand palette onto the canvas, connect it to other stages with arrows, and configure each stage's properties in a side panel.

Plugin

A pre-built, configurable unit of pipeline logic. Plugins fall into a few categories — source, transform, sink, aggregate, join, action, error handler — described in full in Section 5. Google, Cask, and third parties publish plugins to the Hub.

Hub

Cloud Data Fusion's in-product catalog of installable plugins and reusable pipeline templates. Browsing the Hub is usually the fastest way to find a connector for a given source or destination system.

Wrangler

The interactive data preparation tool: you point it at a small sample of your data, see it rendered as a spreadsheet-like grid, and clean/reshape it by clicking columns and choosing operations. Every operation is recorded as a human-readable **directive** (Section 6), and the resulting recipe can be exported directly into a pipeline as a Wrangler transform stage.

Directive

A single, named data-transformation step recorded by Wrangler — for example parse-as-csv or uppercase :status. A sequence of directives is called a **recipe**, and Wrangler recipes are just plain text, which makes them easy to read, version, and reuse.

Schema

The explicit list of field names and types flowing between two connected stages. Cloud Data Fusion propagates and validates schema at every connection on the canvas, catching type mismatches at design time rather than at pipeline run time.

Runtime argument / Macro

A placeholder value (written ${argument_name}) used in a plugin's configuration instead of a hardcoded value, so the same deployed pipeline can run against different values — a different date, table name, or file path — on each run without editing the pipeline itself.

Compute profile

The configuration that tells Cloud Data Fusion what kind of Spark cluster to provision for a pipeline run — machine type, worker count, autoscaling — and whether that cluster is ephemeral (created fresh per run) or an existing cluster you point it at.

Managed Service for Apache Spark (formerly Dataproc)

The underlying managed Spark service that actually executes most Cloud Data Fusion pipeline runs. As of April 2026, Google rebranded Dataproc (and Google Cloud Serverless for Apache Spark) under this unified name; the API, CLI commands, and IAM role names still say "dataproc" internally. Cloud Data Fusion typically provisions a short-lived cluster for each pipeline run and tears it down afterward, so you are not paying for idle compute between runs.

Private IP instance

A Cloud Data Fusion instance configured so its execution environment lives inside your VPC on a private IP, with no public endpoint for the data plane — the standard choice once you're doing more than a quick demo.

Data lineage

Cloud Data Fusion's automatically tracked record of which datasets fed into which pipelines and which fields fed into which output fields, viewable in the product's Lineage graph — useful for impact analysis and governance.

Replication job

A separate Cloud Data Fusion capability (distinct from a batch/streaming pipeline) purpose-built for continuously replicating an entire database into BigQuery, using the same underlying change-capture mechanism as Datastream.

3. How Cloud Data Fusion Works Under the Hood

3.1 Two moving parts: the control plane and the execution plane

It helps to think of every Cloud Data Fusion instance as having two separate layers:

- **Control plane (the CDAP application)** — a Google-managed deployment (running on GKE) that hosts the UI, the pipeline definitions, the Hub, Wrangler, scheduling, and metadata. This is always running once your instance exists, which is why instances are billed by the hour they're up, not by how much data they've processed.
- **Execution plane (Spark clusters)** — the actual compute that runs a pipeline. For most pipelines, Cloud Data Fusion provisions an ephemeral Managed Service for Apache Spark (Dataproc) cluster when a run starts, executes the compiled Spark job, and deletes the cluster when the run finishes.

> **Why this two-layer design matters for cost**
> Deleting or stopping your Data Fusion instance stops the control-plane billing. But if you leave the instance running and idle with no pipelines executing, you're only paying the instance's hourly design cost — execution cost (the Spark clusters) is separate and only incurred while a pipeline is actually running. Section 8.2 breaks this down further.

3.2 From a canvas to a running Spark job

1. You drag plugins onto the Pipeline Studio canvas and connect them — this defines a directed acyclic graph (DAG) of stages.
2. On **Deploy**, Cloud Data Fusion validates the schema flowing through every connection and compiles the DAG into an executable Spark application (for batch) or a Spark Structured Streaming application (for streaming pipelines).
3. On **Run**, the control plane requests a Spark cluster from Managed Service for Apache Spark (creating one if using an ephemeral compute profile), submits the compiled job to it, and streams logs and metrics back into the CDAP UI while it runs.
4. When the run finishes (or, for streaming pipelines, when you stop it), an ephemeral cluster is torn down automatically.

3.3 Batch vs. streaming pipelines

Pipeline type	Behavior	Typical sources
Batch	Runs to completion once triggered (manually, on a schedule, or by another pipeline's completion), then stops.	Cloud Storage files, database tables, BigQuery, SaaS API extracts.

Pipeline type	Behavior	Typical sources
Streaming	Runs continuously, processing records as they arrive, until explicitly stopped.	Pub/Sub topics, Kafka topics, other continuously-arriving feeds.

3.4 How data reaches Cloud Data Fusion

Because pipelines execute as Spark jobs, connectivity is governed by whatever network path the underlying Spark cluster has: for sources inside Google Cloud (Cloud Storage, BigQuery, Cloud SQL) this is usually straightforward IAM-based access; for external or on-premises databases, the same connectivity patterns used elsewhere on Google Cloud apply — a JDBC connection over a VPN/Interconnect, or a forward proxy — configured per connection in the plugin's properties.

4. Editions and Instance Sizing

Cloud Data Fusion instances come in three editions, which primarily differ in scale, availability, and enterprise features rather than core pipeline-building capability:

Edition	Best for	Notable characteristics
Developer	Learning, demos, small individual projects.	Lowest hourly cost; single-node CDAP deployment; limited to a small number of concurrent accounts; no high availability.
Basic	Standard production workloads at moderate scale.	First 120 hours per month are free per Cloud account; supports more concurrent users and pipelines than Developer.
Enterprise	Large-scale, mission-critical integration.	Adds high availability, private IP by default, higher concurrency limits, and access to the full plugin catalog including premium/enterprise connectors (e.g. SAP).

Demo note
The walkthrough in Section 7 uses a **Developer** edition instance — the cheapest option and more than sufficient for learning the tool on a small CSV file. Switch to Basic or Enterprise once you're building something that needs to run unattended in production, be highly available, or be reached only over a private network.

You are billed for a Cloud Data Fusion instance for every hour it exists and is running, regardless of whether a pipeline is actively executing — so for training or demo purposes, delete the instance as soon as you're done (Section 7's cleanup step covers this).

5. The Building Blocks: Sources, Transforms, Sinks, and More

Every stage you drag onto the Pipeline Studio canvas is a plugin belonging to one of these categories:

Category	Role	Examples
Source	Reads data into the pipeline.	Cloud Storage, BigQuery, Cloud SQL, JDBC (generic databases), Pub/Sub, Kafka, SAP, Salesforce.
Transform	Reshapes, cleans, or enriches records one at a time.	Wrangler, JavaScript/Python (custom code), Projection (select/rename/cast fields), Data Masking.
Aggregate	Groups and summarizes across many records — Cloud Data Fusion's equivalent of SQL's GROUP BY.	Group By, Deduplicate.
Join	Combines records from two or more input branches on a key.	Joiner (inner/outer/broadcast join).
Sink	Writes data out of the pipeline.	BigQuery, Cloud Storage, Cloud SQL, Pub/Sub, JDBC.
Error handler / error collector	Catches records that failed validation or transformation elsewhere in the pipeline, so bad rows don't silently vanish or crash the whole run.	Error Collector, writing rejects to a separate "quarantine" sink.
Action	A step that isn't record-by-record data flow — running before or after the main pipeline.	Running a script, sending an email/notification, triggering another pipeline.
Condition	Branches the pipeline based on a boolean expression, e.g. only continuing if an upstream action succeeded.	Used with Action stages for simple orchestration logic.

5.1 When a plugin doesn't exist for what you need

Two escape hatches exist without leaving the visual model entirely:

- **JavaScript/Python transform plugins** — write a small snippet of code that receives each record and returns a transformed one, embedded as a single stage on the canvas. This is the pressure-release valve for logic too specific for an off-the-shelf plugin.
- **Custom plugins** — package your own plugin as a JAR (for JVM-based plugins) and upload it to your instance's Hub, if you have logic you'll reuse across many pipelines.

6. Wrangler: Interactive Data Preparation

Wrangler is the part of Cloud Data Fusion most people learn first, because it turns data cleaning into something visual and immediate rather than something you write blind. You connect it to a sample of your data — a prefix in Cloud Storage, a BigQuery table, an uploaded file — and it renders a preview grid, similar to a spreadsheet, with one row per record and one column per field.

6.1 How directives work

Every cleaning operation you perform — right-clicking a column and choosing "Uppercase", or typing a transformation into the command line at the top of the grid — is recorded as a **directive**: a short, readable line of text. A sequence of directives is a **recipe**, and because it's just text, a recipe is easy to read back later, share, or check into version control alongside the rest of a pipeline's definition.

```
# A short example recipe, as Wrangler would record it while you click through
cleaning
# a raw CSV export:

parse-as-csv :body ',' true
drop :body
set-headers :order_id,:customer_name,:amount,:status
set-type :amount double
filter-rows-on condition-false amount > 0
uppercase :status
fill-null-or-empty :status 'UNKNOWN'
```

What that recipe does, line by line

Directive	What it does
parse-as-csv :body ',' true	Splits the raw text field into columns on comma delimiters, treating the first row as a header.
drop :body	Removes the now-redundant original raw text column.
set-headers ...	Explicitly names the resulting columns.
set-type :amount double	Casts the amount column from text to a numeric type.
filter-rows-on condition-false amount > 0	Drops rows that fail the condition — here, non-positive amounts.
uppercase :status	Normalizes the status column's casing.
fill-null-or-empty :status 'UNKNOWN'	Replaces missing status values with a default so downstream logic doesn't have to handle nulls.

6.2 From Wrangler to a pipeline

Once a recipe looks right against the sample data, clicking **Create Pipeline** in Wrangler generates a new pipeline in Pipeline Studio with a source stage pointed at the data you were wrangling and a Wrangler transform stage pre-loaded with your recipe — ready for you to attach a sink and any further stages. This is the exact path the demo in Section 7 follows.

7. Hands-On Demo: CSV in Cloud Storage to a Clean BigQuery Table

This demo builds a small, real pipeline: a raw, slightly messy CSV of orders lands in Cloud Storage, you clean it interactively with Wrangler, turn that into a deployed pipeline with a BigQuery sink, run it, and verify the result with a query. Budget about 45–60 minutes, most of it spent waiting for the Data Fusion instance to provision (this alone can take 15–30 minutes) and for the first pipeline run's ephemeral Spark cluster to spin up.

Step 1 — Enable the required APIs

```
gcloud services enable \
datafusion.googleapis.com \
storage.googleapis.com \
bigquery.googleapis.com \
dataproc.googleapis.com \
--project=$PROJECT_ID
```

Why dataproc.googleapis.com
Even though the product is now branded Managed Service for Apache Spark, the API, and the permissions your Data Fusion service account needs, are still named dataproc — this is expected (Section 2's note on the rebrand).

Step 2 — Create a small demo CSV and upload it to Cloud Storage

Save the following as orders_raw.csv — note the deliberately messy casing and one bad row, so the Wrangler steps below have something real to fix:

```
order_id,customer_name,amount,status
1001,Asha Rao,1499.00,placed
1002,Vikram Iyer,799.50,PLACED
1003,Meera Nair,-50.00,cancelled
1004,Rohan Das,2199.00,shipped
1005,Divya Menon,,placed
```

```
gcloud storage buckets create gs://$PROJECT_ID-datafusion-demo \
--location=us-central1 \
--uniform-bucket-level-access

gcloud storage cp orders_raw.csv \
gs://$PROJECT_ID-datafusion-demo/raw/orders_raw.csv
```

Step 3 — Create a Developer-edition Cloud Data Fusion instance

```
gcloud data-fusion instances create cdf-demo \
--location=us-central1 \
--edition=developer \
--version=6.11 \
--project=$PROJECT_ID
```

> **This step is slow**
> Instance creation commonly takes 15–30 minutes. This is a good point to review Sections 5 and 6 again while you wait, or start the demo before a training session so it's ready when the room is.

Step 4 — Grant the Data Fusion service account the permissions it needs

Cloud Data Fusion runs pipelines using a Google-managed service agent that needs permission to launch Spark clusters and access your project's data. In the console, under **IAM & Admin** → **IAM**, confirm the service account service-<PROJECT_NUMBER>@gcp-sa-datafusion.iam.gserviceaccount.com has the **Cloud Data Fusion API Service Agent** role, and that the default Compute Engine service account has at least **Editor** (or narrower, purpose-built roles for BigQuery, Cloud Storage, and Dataproc, if you prefer to scope this down for anything beyond a demo).

Step 5 — Open the Cloud Data Fusion UI

In the console, go to **Data Fusion** → **Instances**, wait for cdf-demo to show a green "Running" status, then click **View Instance**. This opens the CDAP web UI — everything from here happens inside that UI rather than via gcloud.

Step 6 — Wrangle the raw CSV

1. From the CDAP UI left navigation, choose **Wrangler**.
2. Click **Add Connection** → **Cloud Storage** (or use the pre-configured GCS connection), and browse to gs://<PROJECT_ID>-datafusion-demo/raw/orders_raw.csv.
3. Wrangler loads a preview grid with a single column of raw text — click the column's dropdown and choose **Parse** → **CSV**, treating the first row as a header. This is exactly the parse-as-csv directive from Section 6.1.
4. Set the amount column's type to a decimal/double via its column dropdown (**Change data type**).
5. Right-click the status column and apply **Uppercase**, so "placed"/"PLACED" become consistent.
6. Use the Wrangler command line to add a row filter that drops non-positive amounts: type filter-rows-on condition-false amount > 0 and press enter.
7. Add one more directive to handle the missing status in the last row: fill-null-or-empty :status 'UNKNOWN'.
8. Confirm the recipe panel on the right now lists all of the directives you just applied, in order — this is your reusable, readable recipe from Section 6.1.

Step 7 — Turn the wrangled recipe into a pipeline

1. Click **Create Pipeline** in the top right of the Wrangler screen, and choose **Batch pipeline**.
2. Pipeline Studio opens with two connected stages already placed: a **GCSFile** source pointed at your CSV, and a **Wrangler** transform pre-loaded with your recipe.
3. From the plugin palette on the left, under **Sink**, drag a **BigQuery** stage onto the canvas.
4. Draw a connection arrow from the Wrangler stage's output to the BigQuery stage's input.
5. Click the BigQuery stage to configure it: set the dataset (e.g. datafusion_demo — Cloud Data Fusion will create it if it doesn't exist) and table name (e.g. orders_clean), and leave the write mode at its default (create/append).

Step 8 — Deploy and run

1. Click **Deploy** in the top right. Cloud Data Fusion validates the schema across every connection on the canvas — this is where a type mismatch would be caught before you spend time waiting on a cluster.
2. Click **Run**. Watch the pipeline's status move from *Provisioning* (an ephemeral Spark cluster is being created) to *Running* to *Succeeded*.
3. While it runs, click into any stage to see live record counts flowing through it — a quick way to sanity-check that, for example, exactly one row (the negative amount) was dropped by your filter directive.

> **First run is the slowest**
> The very first run of a demo pipeline is usually the slowest one, because an ephemeral Spark cluster has to be provisioned from scratch. Subsequent runs against a warm compute profile (if configured) are faster.

Step 9 — Verify the result in BigQuery

```
bq query --use_legacy_sql=false \
'SELECT * FROM `'"$PROJECT_ID"'.datafusion_demo.orders_clean` ORDER BY order_id'
```

You should see four rows — order 1003 (the negative amount) is gone, every status value is upper-case, and order 1005's missing status now reads UNKNOWN — exactly what the recipe from Step 6 specified, now running as a repeatable, deployed pipeline rather than a one-off manual edit.

Step 10 — Try it again with a schedule or a runtime argument (optional)

From the deployed pipeline's detail page, **Schedule** lets you attach a time-based trigger so it runs automatically. To make the pipeline reusable rather than hardcoded to this one file, edit the GCSFile source's path property to use a runtime argument such as gs://${bucket}/raw/${filename} (Section 2's definition of macros), then supply bucket and filename values each time you trigger a run.

Step 11 — Clean up

Cloud Data Fusion instances are billed hourly while they exist, whether or not a pipeline is running (Section 3.1). Delete the demo instance and its supporting resources when you're done:

```
gcloud data-fusion instances delete cdf-demo \
--location=us-central1 --project=$PROJECT_ID --quiet

bq rm -r -f -d $PROJECT_ID:datafusion_demo
gcloud storage rm -r gs://$PROJECT_ID-datafusion-demo
```

8. Operations: Monitoring, Cost, Security

8.1 Monitoring a pipeline

- Every pipeline run has a detail page showing per-stage record counts, a live log stream, and a final status (Succeeded/Failed/Killed) — the first place to look after any run.
- The **Monitor** section of the CDAP UI aggregates run history, duration trends, and failure rates across all pipelines in the instance.
- Cloud Monitoring also receives Data Fusion metrics, which can be wired into alerting policies the same way as other Google Cloud services.

8.2 Cost shape

Cost driver	Notes
Instance design cost	Billed per hour the instance exists and is running, regardless of pipeline activity — Basic edition includes 120 free hours/month per account.
Pipeline execution cost	Billed for the Managed Service for Apache Spark (Dataproc) cluster resources consumed while a pipeline actually runs — an idle, running instance with no active pipelines costs only the design cost above.
Storage & warehouse costs	Ordinary Cloud Storage and BigQuery storage/query costs for whatever a pipeline reads and writes.

8.3 Security basics

- For anything beyond a demo, use a **private IP instance** so pipeline execution stays inside your VPC rather than traversing the public internet.
- Scope the Data Fusion service agent and the Dataproc-executing service account to the minimum roles needed for the specific sources/sinks in use, rather than broad Editor access.
- Enterprise edition adds Customer-Managed Encryption Keys (CMEK) and VPC Service Controls support for organizations with stricter compliance requirements.

9. Best-Practice Checklist

1. Prototype transformations in Wrangler against a small sample before wiring up a full pipeline — it's far faster to iterate on a recipe than to redeploy a pipeline repeatedly.
2. Give every pipeline an Error Collector (or equivalent) path so malformed records are captured and inspectable rather than silently dropped or crashing the run.
3. Use runtime arguments/macros for anything that changes between runs (file paths, table names, dates) instead of hardcoding values into a deployed pipeline.
4. Keep an eye on instance edition vs. actual need — don't default to Enterprise for a workload a Basic instance comfortably handles.
5. Delete or pause Developer/demo instances when not actively in use, since the design cost accrues hourly regardless of pipeline activity.
6. Use Private IP instances and scoped IAM roles for anything beyond a learning exercise.
7. Check the Lineage view before making a breaking schema change to a widely-used source dataset, to see what pipelines depend on it.

10. Troubleshooting Guide

Symptom	Likely cause	What to check
Instance stuck "Creating" for a long time	Normal — instance creation routinely takes 15–30 minutes.	Wait; check the Data Fusion instance's operation status in the console before assuming failure.
Pipeline fails at "Provisioning"	The Data Fusion / Dataproc service accounts lack permission to create a Spark cluster.	Confirm the IAM roles from Step 4 of the demo are correctly assigned.
Deploy fails with a schema mismatch error	Two connected stages disagree on a field's type or name.	Open the connection's schema view and reconcile the mismatched field before redeploying.
Wrangler preview shows unexpected blank/garbled columns	The source wasn't parsed with the right format/delimiter yet.	Re-check the parse-as-csv (or equivalent) directive's delimiter and header settings.
Pipeline run succeeds but the destination table is empty	A filter directive or downstream condition dropped every row.	Check per-stage record counts on the run detail page to see where the row count dropped to zero.
BigQuery sink fails with a permissions error	The pipeline's executing service account lacks BigQuery Data Editor / Job User roles.	Grant the missing roles to the Dataproc-executing service account, not just the Data Fusion service agent.

11. Appendix: Full Glossary & Further Reading

Full glossary (quick reference)

Term	Plain-English meaning
CDAP	The open-source platform Cloud Data Fusion is built on.
Instance	A dedicated Cloud Data Fusion deployment within your project.
Namespace	A logical grouping of pipelines/datasets within an instance.
Pipeline	A connected graph of stages defining a batch or streaming data flow.
Pipeline Studio	The visual canvas for building pipelines.
Plugin	A pre-built, configurable unit of pipeline logic (source/transform/sink/etc.).
Hub	The in-product catalog of installable plugins and pipeline templates.
Wrangler	The interactive, spreadsheet-like data preparation tool.
Directive / Recipe	A single Wrangler operation, and an ordered sequence of them.
Schema	The typed field list validated at every connection on the canvas.
Runtime argument / Macro	A placeholder value resolved at pipeline run time.
Compute profile	The configuration for what Spark cluster a pipeline run uses.
Managed Service for Apache Spark	The rebranded name for Dataproc; executes pipeline runs.
Private IP instance	An instance whose execution plane has no public endpoint.
Data lineage	The tracked record of dataset/field dependencies across pipelines.
Replication job	A dedicated whole-database-to-BigQuery replication capability.

Where to go next

- Cloud Data Fusion documentation — official docs, the source of truth for current editions, quotas, and the full plugin list.
- The Hub, inside any running instance — the fastest way to see what connectors are actually available for your specific source and destination systems.
- The companion handbook, *Datastream + Landing Zone Patterns for CDC*, for the log-based change-capture approach Cloud Data Fusion's Replication feature also builds on.

End of handbook. Section 7's demo is self-contained — start from Step 1 whenever you're ready to try it yourself.